

# Docker-02

## Task:1

1.Create a tomcat container on 8080 and deploy sample application in tomcat.

Step :1

—> docker pull tomcat

\* pull tomcat and run the image on 8080

```
Feb 05 09:10:00 ip-172-31-20-146.eu-north-1.compute.internal systemd[1]: Started docker.service - Docker Application Container Engine.
[root@ip-172-31-20-146 ~]# docker pull tomcat
Using default tag: latest
latest: Pulling from library/tomcat
a3629ac5b9f4: Pull complete
bd41d65c3828: Pull complete
8be760c2a9a8: Pull complete
4f4fb700ef54: Pull complete
d0702afe109d: Pull complete
d99c7608407b: Pull complete
2ca1458a9031: Pull complete
Digest: sha256:6af1184ab272d631d62a712260ac858010a898b835c088cbe447653cdc1d3d97
Status: Downloaded newer image for tomcat:latest
docker.io/library/tomcat:latest
[root@ip-172-31-20-146 ~]# docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
jenkins/jenkins     latest          64cb03009c23   41 hours ago   487MB
tomcat               latest          fe1a24e4a16c   9 days ago     412MB
[root@ip-172-31-20-146 ~]# docker run -itd -p 8080:8080 tomcat
96110eb6ee840b54224b1af34fe8d1d045f1cc26d74e2e789daa8de6e8cb576f
[root@ip-172-31-20-146 ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
96110eb6ee84   tomcat   "catalina.sh run"        16 seconds ago Up 15 seconds  0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   keen_volhard
[root@ip-172-31-20-146 ~]#
```

—> docker images

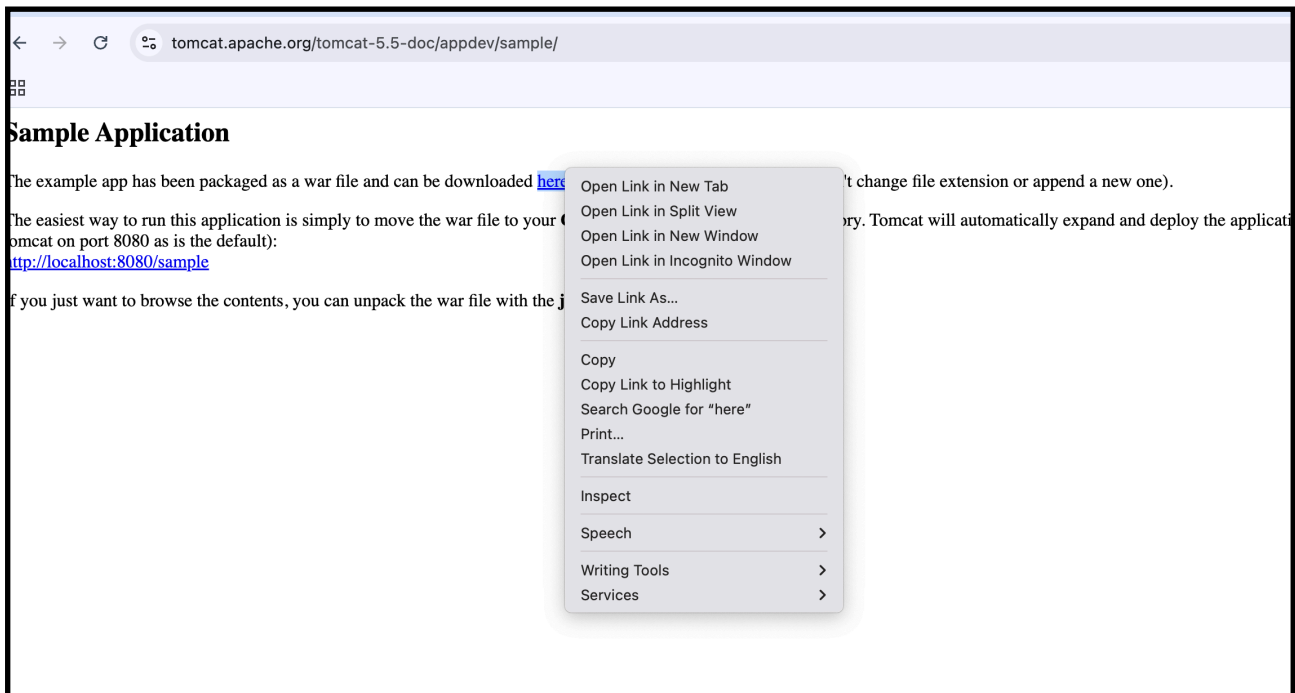
—> docker run -itd -p 8080:8080 tomcat

—> docker ps

Step:2

—> Deploy Sample Application (WAR file)

\* go to official tomcat page of sample war



—> copy the link address

\* now Use this command to go in to tomcat container

—> docker exec -it 96110eb6ee84 bash

```
[root@ip-172-31-20-146 ~]# docker exec -it 96110eb6ee84 bash
root@96110eb6ee84:/usr/local/tomcat# ls
bin BUILDING.txt conf CONTRIBUTING.md filtered-KEYS lib LICENSE logs native-jni-lib NOTICE README.md RELEASE-NOTES RUNNING.txt temp
root@96110eb6ee84:/usr/local/tomcat# cd webapps
root@96110eb6ee84:/usr/local/tomcat/webapps# ls
```

—> cd webapps

## Step:3

—> now use wget <link address which we copied >

--> if you get the wget command not found then

—> apt update -y

—> apt install wget -y

```
root@96110eb6ee84:/usr/local/tomcat/webapps# wget https://tomcat.apache.org/tomcat-5.5-doc/appdev/sample/sample.war
bash: wget: command not found
root@96110eb6ee84:/usr/local/tomcat/webapps# apt update -y
Get:1 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [524 B]
```

```
root@96110eb6ee84:/usr/local/tomcat/webapps# apt install wget -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libpsl5t64 publicsuffix
The following NEW packages will be installed:
  libpsl5t64 publicsuffix wget
0 upgraded, 3 newly installed, 0 to remove and 6 not upgraded.
Need to get 520 kB of archives.
After this operation, 1,423 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 libpsl5t64 amd64 0.21.2-1.1build1 [57.1 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 publicsuffix all 20231001.0357-0.1 [129 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 wget amd64 1.21.4-1ubuntu4.1 [334 kB]
Fetched 520 kB in 1s (660 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
```

—> now wget <url>

```
root@96110eb6ee84:/usr/local/tomcat/webapps# wget https://tomcat.apache.org/tomcat-5.5-doc/appdev/sample/sample.war
--2026-02-05 09:18:54-- https://tomcat.apache.org/tomcat-5.5-doc/appdev/sample/sample.war
Resolving tomcat.apache.org (tomcat.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to tomcat.apache.org (tomcat.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4606 (4.5K)
Saving to: 'sample.war'

sample.war                                     100%[=====]

2026-02-05 09:18:54 (102 MB/s) - 'sample.war' saved [4606/4606]

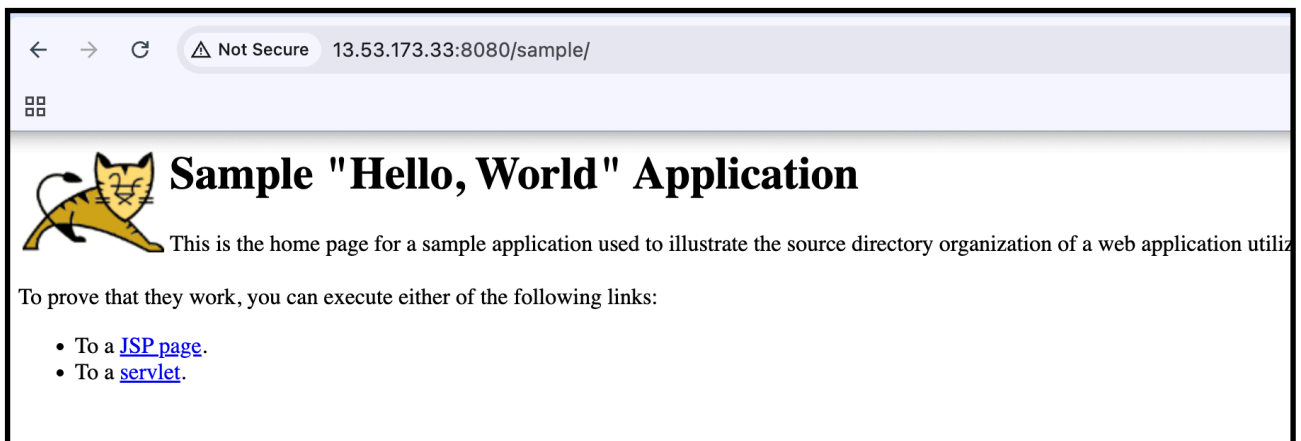
root@96110eb6ee84:/usr/local/tomcat/webapps#
```

—> ls

```
[root@96110eb6ee84:/usr/local/tomcat/webapps# ls
sample sample.war
[root@96110eb6ee84:/usr/local/tomcat/webapps# ll
total 8
drwxr-xr-x. 1 root root 38 Feb 5 09:18 ./
drwxr-xr-x. 1 root root 57 Jan 26 22:13 ../
drwxr-x---. 5 root root 86 Feb 5 09:18 sample/
-rw-r--r--. 1 root root 4606 Mar 31 2012 sample.war
root@96110eb6ee84:/usr/local/tomcat/webapps#
```

Step :4

—> now check your public ip:8080/sample



## Task:2

2.Create volume and deploy tomcat container on port 8081.

### 1. Create a Docker Volume

```
docker volume create tomcat-data
```

This creates a named volume tomcat-data.

/var/lib/docker/volumes

```

exit
[root@ip-172-31-20-146 ~]# docker volume create tomcat-data
tomcat-data
[root@ip-172-31-20-146 ~]# cd /var/lib/docker/
[root@ip-172-31-20-146 docker]# ls
buildkit containers engine-id image network overlay2 plugins runtimes swarm tmp volumes
[root@ip-172-31-20-146 docker]# cd volumes/
[root@ip-172-31-20-146 volumes]# ls
259eeff72e272006b82922b8b653a46325a168f111d75d7ab524388c1177b26c 87752aa0f3145b9a1cebc003296651911a6012284d744a1ddd574bac423dca1e backingFsBlockDev metadata.db tomcat-data
[root@ip-172-31-20-146 volumes]# cd tomcat-data/
[root@ip-172-31-20-146 tomcat-data]# ls
_data
[root@ip-172-31-20-146 tomcat-data]# cd _data/
[root@ip-172-31-20-146 _data]# ls
[root@ip-172-31-20-146 _data]# pwd
/var/lib/docker/volumes/tomcat-data/_data
[root@ip-172-31-20-146 _data]#

```

## 2. Deploy Sample Application

Inside the /var/lib/docker/volumes/tomcat-data/\_data

Deploy the sample war file in \_data

--> Wget <url>

```

[root@ip-172-31-20-146 _data]# ls
[root@ip-172-31-20-146 _data]# pwd
/var/lib/docker/volumes/tomcat-data/_data
[root@ip-172-31-20-146 _data]# wget https://tomcat.apache.org/tomcat-5.5-doc/appdev/sample/sample.war
--2026-02-05 09:34:12-- https://tomcat.apache.org/tomcat-5.5-doc/appdev/sample/sample.war
Resolving tomcat.apache.org (tomcat.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to tomcat.apache.org (tomcat.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4606 (4.5K)
Saving to: 'sample.war'

sample.war                                     100%[=====]

2026-02-05 09:34:12 (102 MB/s) - 'sample.war' saved [4606/4606]

[root@ip-172-31-20-146 _data]# ls
sample.war
[root@ip-172-31-20-146 _data]# ll
total 8
-rw-r--r--. 1 root root 4606 Mar 31 2012 sample.war
[root@ip-172-31-20-146 _data]# ll
total 8
-rw-r--r--. 1 root root 4606 Mar 31 2012 sample.war
[root@ip-172-31-20-146 _data]#

```

## 3. Run Tomcat Container on Port 8083 with Volume

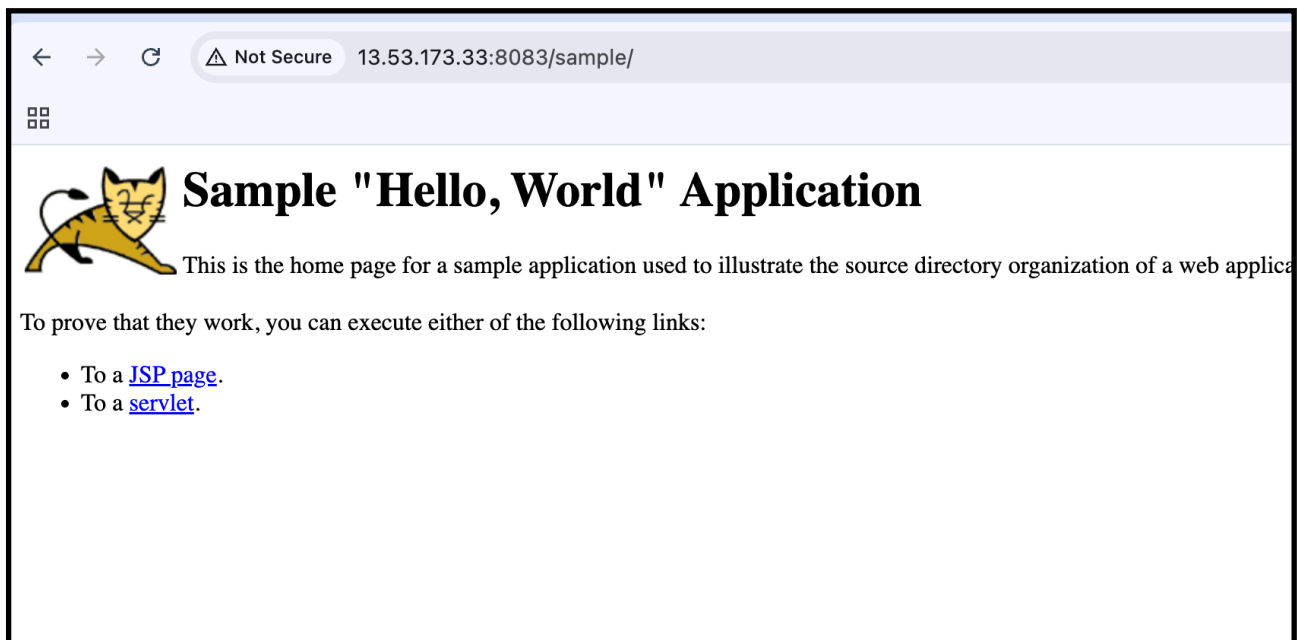
docker container run -itd -p 8083:8080 -v /var/lib/docker/volumes/tomcat-data/\_data:/usr/local/tomcat/webapps tomcat:latest

```

[root@ip-172-31-20-146 _data]# docker container run -itd -p 8083:8080 -v /var/lib/docker/volumes/tomcat-data/_data:/usr/local/tomcat/webapps tomcat:latest
cf41d8af6cda0801022af1f75da9b92ecf6b253d71ef79e5d1a467322dbe1661
[root@ip-172-31-20-146 _data]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
cf41d8af6cda   tomcat:latest   "catalina.sh run"        11 seconds ago   Up 11 seconds   0.0.0.0:8083->8080/tcp, :::8083->8080/tcp   hardcore_jones
96110eb6ee84   tomcat         "catalina.sh run"        25 minutes ago   Up 25 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   keen_volhard
[root@ip-172-31-20-146 _data]# ll
total 8
drwxr-x---. 5 root root   86 Feb  5 09:36 sample
-rw-r--r--. 1 root root 4606 Mar 31 2012 sample.war
[root@ip-172-31-20-146 _data]#

```

—-> now check your public ip:8083/sample



## Task:3

3.Limit the nginx container to 500 MB.

`docker run -itd -p 81:80 --memory=500m nginx`

—> What's happening here

-i -t -d → interactive + tty + detached (works, but -it isn't really needed for nginx)

-p 81:80 → host port 81 → container port 80

--memory=500m → limits RAM to 500 MB

nginx → image

```
[root@ip-172-31-20-146 ~]# docker run -itd -p 81:80 --memory=500m nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
0c8d55a45c0d: Pull complete
46bf3a120c8e: Pull complete
4f4efe02d542: Pull complete
7b6cb8ccac7b: Pull complete
f73400a233fd: Pull complete
47cd406a84ef: Pull complete
bae5a1799a80: Pull complete
Digest: sha256:b17697e86d0c02378716277d09f45b946f8709aaa12c708e30fdd4736f536af1
Status: Downloaded newer image for nginx:latest
d283d67a9ebdab89831c243d3135a556fc93ffdc8c5a7f09ee0030c8f12b46f5
[root@ip-172-31-20-146 ~]# docker ps
```

| CONTAINER ID | IMAGE         | COMMAND                 | CREATED        | STATUS        | PORTS                                     | NAMES           |
|--------------|---------------|-------------------------|----------------|---------------|-------------------------------------------|-----------------|
| d283d67a9ebd | nginx         | "/docker-entrypoint..." | 12 seconds ago | Up 11 seconds | 0.0.0.0:81->80/tcp, :::81->80/tcp         | serene_roentgen |
| cf41d8af6cda | tomcat:latest | "catalina.sh run"       | 8 minutes ago  | Up 8 minutes  | 0.0.0.0:8083->8080/tcp, :::8083->8080/tcp | hardcore_jones  |
| 96110eb6ee84 | tomcat        | "catalina.sh run"       | 33 minutes ago | Up 33 minutes | 0.0.0.0:8080->8080/tcp, :::8080->8080/tcp | keen_volhard    |

—> docker stats

```
[root@ip-172-31-20-146 ~]# docker stats
```

| CONTAINER ID | NAME            | CPU % | MEM USAGE / LIMIT   | MEM % | NET I/O         | BLOCK I/O   | PIDS |
|--------------|-----------------|-------|---------------------|-------|-----------------|-------------|------|
| d283d67a9ebd | serene_roentgen | 0.00% | 3.355MiB / 500MiB   | 0.67% | 726B / 0B       | 0B / 32.8kB | 3    |
| cf41d8af6cda | hardcore_jones  | 0.10% | 98.5MiB / 7.601GiB  | 1.27% | 4.28kB / 5.62kB | 0B / 221kB  | 35   |
| 96110eb6ee84 | keen_volhard    | 0.05% | 157.4MiB / 7.601GiB | 2.02% | 36.5MB / 127kB  | 0B / 64.5MB | 35   |

## Task :4 (task 5 - task 10)

4.Create a sample docker file using below instructions.

1. Base module as amazonlinux:latest. Task 5
2. Maintainer you name task 6
3. Install nginx task 7
4. COPY one index.html file to image. Task 8
5. Expose on port 80 task 9
6. Command to start the nginx container task 10

## Step :1

### Requirements :

1. Create a file named **Dockerfile** with the above contents.

Vi docker file

2.

### Create **index.html**

```
[root@ip-172-31-20-146 ~]# vi index.html
[root@ip-172-31-20-146 ~]# cat
.bash_history .bash_logout .bash_profile .bashrc
[root@ip-172-31-20-146 ~]# cat index.html
<!DOCTYPE html>
<html>
<head>
  <title>Custom Nginx</title>
</head>
<body>
  <h1>Hello from Amazon Linux Nginx</h1>
</body>
</html>

[root@ip-172-31-20-146 ~]# █
```



## Vi Dockerfile

```
[root@ip-172-31-20-146 ~]# vi dockerfile
[root@ip-172-31-20-146 ~]# cat dockerfile
# 5. Base module as amazonlinux:latest
FROM amazonlinux:latest

# 6. Maintainer name
MAINTAINER Waseem Akram

# 7. Install nginx
RUN yum update -y && \
    yum install -y nginx && \
    yum clean all

# 8. Copy one index.html file to image
COPY index.html /usr/share/nginx/html/index.html

# 9. Expose on port 80
EXPOSE 80

# 10. Command to start the nginx container
CMD ["nginx", "-g", "daemon off;"]

[root@ip-172-31-20-146 ~]# █
```

```
# 5. Base module as amazonlinux:latest
FROM amazonlinux:latest

# 6. Maintainer name
MAINTAINER Waseem Akram

# 7. Install nginx
RUN yum update -y && \
    yum install -y nginx && \
    yum clean all

# 8. Copy one index.html file to image
COPY index.html /usr/share/nginx/html/index.html

# 9. Expose on port 80
EXPOSE 80

# 10. Command to start the nginx container
CMD ["nginx", "-g", "daemon off;"]
█
~
~
~
~
~
```

## Step 2: Build the Docker image

```
docker build -t amazon-nginx .
```

Check:

```
docker images
```

```
[root@ip-172-31-20-146 ~]# docker build -t amazon-nginx .
[+] Building 18.7s (8/8) FINISHED
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 502B
=> [internal] load metadata for docker.io/library/amazonlinux:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/amazonlinux:latest@sha256:2f10659a297494a1842d8b122027e3d865a7d4e5a88c056e53f45316ffc9c985
=> => resolve docker.io/library/amazonlinux:latest@sha256:2f10659a297494a1842d8b122027e3d865a7d4e5a88c056e53f45316ffc9c985
=> => sha256:2f10659a297494a1842d8b122027e3d865a7d4e5a88c056e53f45316ffc9c985 2.38kB / 2.38kB
=> => sha256:c5e2e0feadeed940289e62b66987612c7b5e00c15ac0c828e0e3ca0538f4203 1.02kB / 1.02kB
=> => sha256:7dd7078330010519b54f7cf0a4f7d785021e507164bd6775775897d5ae89e717 586B / 586B
=> => sha256:0fa079dacd9b36639e4d877eebffd93a115a824e0b36ffbbb73537098b617c1 54.02MB / 54.02MB
=> => extracting sha256:0fa079dacd9b36639e4d877eebffd93a115a824e0b36ffbbb73537098b617c1
=> [internal] load build context
=> => transferring context: 232B
=> [2/3] RUN yum update -y && yum install -y nginx && yum clean all
=> [3/3] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => writing image sha256:5a3a68471a79f053b283f2786d3a11535302c11ca853ee94e28953b63f139eec
=> => naming to docker.io/library/amazon-nginx
[root@ip-172-31-20-146 ~]#
```

```

=> => naming to docker.io/library/amazon-nginx
[ root@ip-172-31-20-146 ~]# docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
amazon-nginx        latest         5a3a68471a79   39 seconds ago 213MB
nginx               latest         5cdef49c3335   10 hours ago   161MB
```

## Step 3: Run the nginx container

```
docker run -d -p 80:80 --name nginx-container amazon-nginx
```

## Step 4: Verify

```
[root@ip-172-31-20-146 ~]# docker run -d -p 80:80 --name nginx-container amazon/nginx
cf284536f3f5caf7a90ca46bac5da83d17e3cee5af85cd38c1fb64a7ccc40d21
[root@ip-172-31-20-146 ~]# docker ps -a
```

| CONTAINER ID | IMAGE        | COMMAND                  | CREATED        | STATUS        | PORTS                             | NAMES            |
|--------------|--------------|--------------------------|----------------|---------------|-----------------------------------|------------------|
| cf284536f3f5 | amazon/nginx | "nginx -g 'daemon of..." | 6 seconds ago  | Up 5 seconds  | 0.0.0.0:80->80/tcp, :::80->80/tcp | nginx-container  |
| d283d67a9ebd | nginx        | "/docker-entrypoint..."  | 20 minutes ago | Up 20 minutes | 0.0.0.0:81->80/tcp, :::81->80/tcp | serene-container |

Browser:

http://<SERVER-IP>



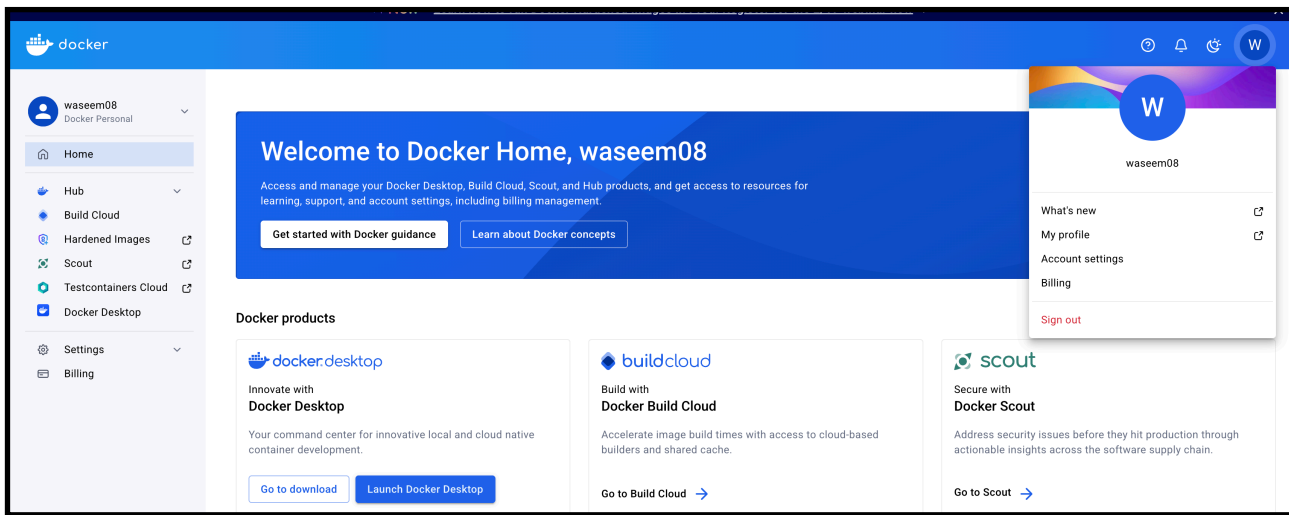
# Task :11

## 11.Push image to Docker hub

### Step 1: Login to Docker Hub

docker login

Enter your **Docker Hub** username and password.



```
60110eb0ce84  commit  cca444d3b1  55 minutes ago  55 minutes ago
[root@ip-172-31-20-146 ~]# docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope
token is recommended for most scenarios.

Username: waseem08
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-20-146 ~]#
```

## Step 2: Tag the image

Your local image:

amazon-nginx

Tag it:

```
docker tag amazon-nginx waseem08/amazon-nginx:latest
```

```
Login Succeeded
[root@ip-172-31-20-146 ~]# docker tag amazon-nginx waseem08/amazon-nginx:latest
[root@ip-172-31-20-146 ~]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
amazon-nginx        latest      5a3a68471a79     13 minutes ago   213MB
waseem08/amazon-nginx latest      5a3a68471a79     13 minutes ago   213MB
```

## Step 3: Push the image

```
docker push waseem08/amazon-nginx:latest
```

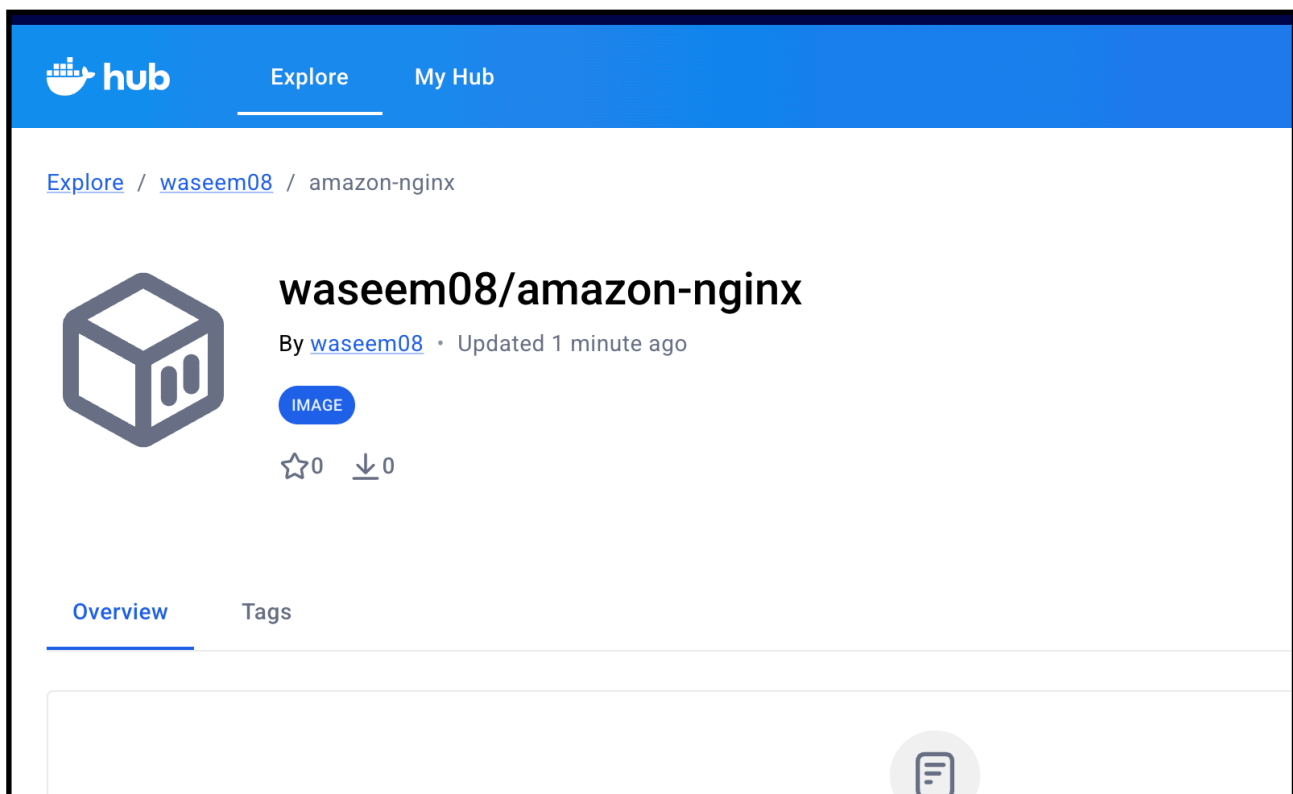
```
tomcat latest fe1a24e4d10c 9 days ago 412MB  
[root@ip-172-31-20-146 ~]# docker push waseem08/amazon-nginx:latest  
The push refers to repository [docker.io/waseem08/amazon-nginx]  
721a997498da: Pushed  
1ae283a870d9: Pushed  
b535d73979ee: Mounted from library/amazonlinux  
latest: digest: sha256:eba6433f9e1b8f2758fd2daa6790b6dffd8b034e1dc5efff5cefea3d348b69a0 size: 948  
[root@ip-172-31-20-146 ~]#
```

## Step 4: Verify

Go to 🖱️ <https://hub.docker.com>

Check **Repositories** → you'll see:

waseem08/amazon-nginx



Explore / waseem08 / amazon-nginx



## waseem08/amazon-nginx

By waseem08 · Updated 2 minutes ago

IMAGE

☆0 ↓0

Manage Repository

Overview **Tags**

Sort by Newest Filter tags

TAG

latest

Last pushed 2 minutes by waseem08

docker pull waseem08/amazon-nginx:latest

| Digest       | OS/ARCH     | Last pull       | Compressed size |
|--------------|-------------|-----------------|-----------------|
| eba6433f9e1b | linux/amd64 | less than 1 day | 73.65 MB        |

# Task:12

## 12.push image to AWS ECR

### STEP 1: Configure AWS

Run:

```
aws configure
```

You'll be asked **4 things**. Fill them like this:

```
AWS Access Key ID      : <your_access_key>
AWS Secret Access Key  : <your_secret_key>
Default region name    : eu-north-1
Default output format  : json
Use the Access Key & Secret Key
```

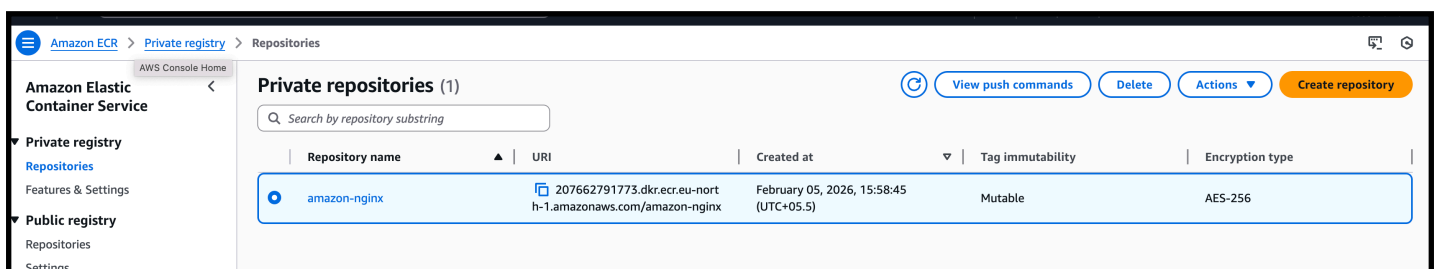
```
AWS Secret Access Key [None]: Fg1SSLEBWX15qrQB21oyp3FuAg
Default region name [None]: eu-north-1
Default output format [None]: json
[root@ip-172-31-20-146 ~]#
```

### STEP 3: Create ECR repository

```
aws ecr create-repository --repository-name amazon-nginx --region eu-north-1
```

```
[root@ip-172-31-20-146 ~]# aws ecr create-repository --repository-name amazon-nginx --region eu-north-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:eu-north-1:207662791773:repository/amazon-nginx",
    "registryId": "207662791773",
    "repositoryName": "amazon-nginx",
    "repositoryUri": "207662791773.dkr.ecr.eu-north-1.amazonaws.com/amazon-nginx",
    "createdAt": "2026-02-05T10:28:45.035000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

--> by this we have created the repository in our aws eu-north-1 region



The screenshot shows the AWS ECR console interface. On the left, there is a navigation menu with 'Amazon Elastic Container Service' and 'Private registry' expanded. The main area is titled 'Private repositories (1)' and contains a table with one repository listed.

| Repository name | URI                                                        | Created at                             | Tag immutability | Encryption type |
|-----------------|------------------------------------------------------------|----------------------------------------|------------------|-----------------|
| amazon-nginx    | 207662791773.dkr.ecr.eu-north-1.amazonaws.com/amazon-nginx | February 05, 2026, 15:58:45 (UTC+05.5) | Mutable          | AES-256         |

At the top right of the console, there are buttons for 'View push commands', 'Delete', 'Actions', and 'Create repository'.

## STEP 4: Login Docker to ECR

```
aws ecr get-login-password --region eu-north-1 | docker login --username AWS --password-stdin 207662791773.dkr.ecr.eu-north-1.amazonaws.com
```

You should see:

Login Succeeded

```
}
[root@ip-172-31-20-146 ~]# aws ecr get-login-password --region eu-north-1 | docker login --username AWS --password-stdin 207662791773.dkr.ecr.eu-north-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-20-146 ~]#
```

## STEP 5: Tag your Docker image

Your local image:

amazon-nginx:latest

Command:

```
docker tag amazon-nginx:latest 207662791773.dkr.ecr.eu-north-1.amazonaws.com/amazon-nginx:latest
```

```
Login Succeeded
[root@ip-172-31-20-146 ~]# docker tag amazon-nginx:latest 207662791773.dkr.ecr.eu-north-1.amazonaws.com/amazon-nginx:latest
[root@ip-172-31-20-146 ~]# docker images
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
207662791773.dkr.ecr.eu-north-1.amazonaws.com/amazon-nginx  latest   5a3a68471a79  43 minutes ago  213MB
amazon-nginx                             latest   5a3a68471a79  43 minutes ago  213MB
```



## STEP 6: Push image to AWS ECR

```
docker push 207662791773.dkr.ecr.eu-north-1.amazonaws.com/  
amazon-nginx:latest
```

```
comcat latest fe1d24e4d16c 9 days ago 412MB  
[root@ip-172-31-20-146 ~]# docker push 207662791773.dkr.ecr.eu-north-1.amazonaws.com/amazon-nginx:latest  
The push refers to repository [207662791773.dkr.ecr.eu-north-1.amazonaws.com/amazon-nginx]  
721a997498da: Pushed  
1ae283a870d9: Pushed  
b535d73979ee: Pushed  
latest: digest: sha256:eba6433f9e1b8f2758fd2daa6790b6dffd8b034e1dc5efff5cefea3d348b69a0 size: 948  
[root@ip-172-31-20-146 ~]#
```

## Verify

AWS Console → ECR → amazon-nginx → Images

—-> now go and check your repository in aws ecr

